

Softbody math

Simulating squish, stretch, and structure

What are softbody physics

Softbody physics are a method of simulating physics by using points and shapes that allow for *squishiness*

- A technique that models objects which can bend, stretch, and deform
- Sits between rigid body physics (no deformation) and full fluid/particle sims (no fixed structure)
- Objects are represented as a mesh of connected points instead of a single rigid shape
- Widely used to simulate materials like rubber, jelly, cloth, muscle, and crashing vehicles
- Trades some physical accuracy for **stability, performance, and control** — good enough physics that looks great and doesn't explode

Examples of games that use softbodies

- **JellyCar** — entire vehicles and terrain are pressurized soft shapes
- **World of Goo** — goo balls connected by springs form structures
- **BeamNG.drive** — node/beam softbody simulation drives vehicle deformation and damage
- **Human: Fall Flat** — floppy, squishy ragdoll-like character movement
- **Poly Bridge, Besiege** — structural stress visualized through soft connections
- Literally every game that does cloth simulation (capas, flags, hair, curtains)

Parts of Softboudys

In the context of 2d games

A note on implementatin

Softbody physics, as a physics simulation method, has many different ways of implementing

- The following method is one such implementation

It follows a fairly **unrealistic**, but **robust and extensible** system

- Useful for most 2D games that want *some* soft body physics without a full research-grade solver
- Priorities, in order:

stability > performance > realism

We'll build it up in layers: points → shapes → collisions → forces → constraints

Points and pointmasses

The core of any 2D (and also 3D) soft body implementation is point masses

These points have:

- *weight* (mass) — affects how much force is needed to move them
- *position* — current location in world space
- *velocity* — current position minus previous position (Verlet-style), or an explicit velocity vector

Example

```
1  class_name PointMass
2
3  var position: Vector2
4  var previous_position: Vector2
5  var mass: float
6  var pinned: bool = false
7
8  func integrate(gravity: Vector2, damping: float) → void:
9      var velocity: Vector2 = (position - previous_position) * damping
10     previous_position = position
11     position += velocity + gravity
```

Example

```
1  class_name PointMass
2
3  var position: Vector2
4  var previous_position: Vector2
5  var mass: float
6  var pinned: bool = false
7
8  func integrate(gravity: Vector2, damping: float) → void:
9      var velocity: Vector2 = (position - previous_position) * damping
10     previous_position = position
11     position += velocity + gravity
```


Example

```
1  class_name PointMass
2
3  var position: Vector2
4  var previous_position: Vector2
5  var mass: float
6  var pinned: bool = false
7
8  func integrate(gravity: Vector2, damping: float) → void:
9      var velocity: Vector2 = (position - previous_position) * damping
10     previous_position = position
11     position += velocity + gravity
```

Shapes

Any shape in a point mass system is simply a connection of points, with lines drawn in a specific order

- A shape = an ordered list of `PointMass` references (usually forming a closed polygon)

A shape's outline can be filled/rendered *independently* of how the physics is calculated (points can drive a mesh, sprite, or line renderer)

Collisions

Making squishy things interact with the world

Finding overlaps

Before we can respond to a collision, we need to *detect* it

Common broad checks: **AABB (bounding box)** or **circle** overlap tests to cheaply rule out far-apart shapes

Once a broad-phase hit is found, do a **narrow-phase** check per point:

- Is this point mass currently inside another shape's polygon?
- Point-in-polygon tests (e.g. ray casting or winding number) are the standard tool

For soft bodies, we check collisions **per point**, not per whole shape

- this is what lets the shape deform on contact instead of bouncing as a rigid unit

Problems with overlaps

Point-in-polygon alone tells you *if* a point is inside, not *how far* or *which way* to push it out

- Fast-moving points can **tunnel** straight through thin shapes between frames

Multiple points overlapping the same shape at once can produce conflicting correction forces

- Corners and concave shapes make "closest edge" ambiguous
- Resolving all overlaps in one pass can re-introduce new overlaps elsewhere (deformation cascades)
- often needs several solver iterations per frame

Raycasting

A ray is a line from an origin in a direction, tested against a shape's edges for intersections

This is how we actually answer "is this point inside that shape?"

The algorithm:

1. Pick a point to test, and cast a single ray from it toward the outside in any fixed direction (e.g. straight right, $(1, 0)$)
2. Check that ray against every edge of the shape, counting how many edges it intersects
 - *Odd number of hits* -> the point is inside the shape
 - *Even number of hits (including zero)* -> the point is outside

Crossing number algorithm

To escape the shape, the ray has to cross its boundary an odd number of times if it started inside, and an even number of times if it started outside

- Edge cases to watch for: the ray passing exactly through a vertex, or running parallel to an edge, both can miscount a hit and need special handling
- Cheap enough to run per point, per frame, which is exactly the granularity soft bodies need for collision

Example

```
1  func is_point_inside(point: Vector2, shape: Array[PointMass]) → bool:
2      var hit_count: int = 0
3
4      for i in shape.size():
5          var a: Vector2 = shape[i].position
6          var b: Vector2 = shape[(i + 1) % shape.size()].position
7
8          if ray_intersects_edge(point, a, b):
9              hit_count += 1
10
11     return hit_count % 2 == 1
```


Example

```
1  func is_point_inside(point: Vector2, shape: Array[PointMass]) → bool:
2      var hit_count: int = 0
3
4      for i in shape.size():
5          var a: Vector2 = shape[i].position
6          var b: Vector2 = shape[(i + 1) % shape.size()].position
7
8          if ray_intersects_edge(point, a, b):
9              hit_count += 1
10
11     return hit_count % 2 == 1
```

Example

```
1  func is_point_inside(point: Vector2, shape: Array[PointMass]) → bool:
2      var hit_count: int = 0
3
4      for i in shape.size():
5          var a: Vector2 = shape[i].position
6          var b: Vector2 = shape[(i + 1) % shape.size()].position
7
8          if ray_intersects_edge(point, a, b):
9              hit_count += 1
10
11     return hit_count % 2 == 1
```

Example

```
1  func is_point_inside(point: Vector2, shape: Array[PointMass]) → bool:
2      var hit_count: int = 0
3
4      for i in shape.size():
5          var a: Vector2 = shape[i].position
6          var b: Vector2 = shape[(i + 1) % shape.size()].position
7
8          if ray_intersects_edge(point, a, b):
9              hit_count += 1
10
11     return hit_count % 2 == 1
```

Forces

What to do with velocity after a collision is *resolved*

Pushing points out of overlap only fixes **position**

- Velocity is still pointing the point right back into the shape
- Left alone, the next frame just recreates the same overlap, causing jitter or points getting "stuck" at the surface

1. *Find the surface normal*

- Use the closest edge found during collision resolution
- Its normal is the direction to correct velocity along

2. *Reflect the velocity*

Springs

ways to keep their shape

A spring connects two points and tries to keep them at a **rest distance** apart

- Basic spring force (Hooke's law): $F = -k * (\text{currentDistance} - \text{restDistance})$
 - k = stiffness - higher values snap back harder and faster

Springs are what turn a "bag of points" into a recognizable, structured shape

Springs

- Typical spring layouts:
 - **Perimeter springs** - connect neighboring outline points, keep the silhouette intact
 - **Cross/internal springs** - connect points across the shape, resist collapsing or turning inside-out
- **Problems with springs:**
 - Too stiff -> jitter and instability, especially at low framerates
 - Too soft -> shape collapses or overstretches under force
 - Long spring chains can feel "floppy" or lag behind fast motion
 - Springs alone don't prevent a shape from folding in on itself

Maintaining shape

filling the objects with "gas", and shape matching

- **Filling with "gas" (pressure model)**
 - treat the shape like an inflated balloon: compute the enclosed area and push outward on the edges to keep that area roughly constant, resisting collapse
- **Shape matching**
 - store an original ("rest") arrangement of points, then each frame nudge current points toward where they'd be if the shape were rigidly rotated/translated to best-fit that original arrangement
 - Gives soft bodies a natural tendency to "remember" their shape and spring back after deformation

Joints and pins

for car wheels

- A *pin* locks a point mass to a fixed position (or to another object), so it never moves under simulation
- useful for anchoring a flag's edge or a rope's end

A *joint* connects two separate soft bodies (or a soft body to a rigid body) at a specific point, allowing rotation but constraining position

Combine pins and joints to build compound structures:

- **Car wheels / vehicle suspension** - soft body "legs" pinned to a rigid chassis at one end, free to flex and absorb impact at the other
- **Ragdolls** - chains of soft or rigid segments jointed together at "elbows" and "knees"
- **Rope and chain** - a line of point masses connected by short, stiff springs, pinned only at the ends

Pins and joints are what let you compose small softbody building blocks into larger, purposeful game objects